

Report Agent — Data Readiness Findings

Cooling Water Pump 1 (CWP-001) · what blocks a diagnostic report, and what no longer does

Prepared 2026-09-08
Sensor f21a66ba-df6d-41b1
9 captures, 720 assessments

The report agent itself is working. It runs all eight steps, produces 36 traceable figures, and passes its own verification — every number in its output resolves to a recorded fact with a stated source. What stops it from being *diagnostic* is the data underneath it, not the code.

Five data problems were identified. **Two are now fixed in code. Three still require measurements that only a person at the plant can take** — not because the software cannot process them, but because the information does not exist anywhere yet. A sixth item I had previously reported as a blocker turns out to have been wrong: that data is already in the database, and connecting to it is the cheapest remaining improvement available.

Item 3 deserves particular attention. It *could* be cleared in about a minute with a database change and no plant visit — and doing so would make the report actively misleading. The distinction between *cannot be done* and *should not be done* matters here, so it is set out in full below.

Fixed — the two that were recoverable

FIXED 1. Shaft speed was missing from every capture

The problem

All 9 captures have `rotation_speed_rpm` empty. Without a shaft speed there are no orders, no bearing fault frequencies, and no velocity — most of vibration diagnosis is unavailable.

What was actually wrong

The speed was never missing. The backend already estimates it from the spectrum and writes `estimated_shaft_hz` into each feature's `metadata`. But `sql_agent/dataset.py` built its rows from a fixed `CSV_COLUMNS` list that did not include `metadata`, so the estimate was discarded before the report agent ever saw it. A transport bug, not a measurement gap.

The fix

Pass `metadata` through. The exported CSV is byte-for-byte unchanged, because the writer is built with `extrasaction="ignore"` — nothing downstream is affected.

Result

1500 rpm (25.0 Hz), agreed by 7 of 8 channels. Channel 3 reports 50.0 Hz, which is exactly twice the consensus — the estimator locked onto the 2X peak there. The report now says so explicitly instead of averaging the disagreement away.

Independent confirmation

The equipment record for CWP-001 gives a nameplate speed of **1480 rpm**. The spectral estimate is within **1.4%** of it. Nothing in the estimator reads the nameplate, so these are two genuinely independent routes to the same answer.

Evidence: `estimated_shaft_hz` = 25.0 on channels 0,1,2,4,5,6,7 and 50.0 on channel 3, read from `measurement_channel_features.metadata`; `equipment_masters.rated_rpm` = 1480 for CWP-001.

FIXED 2. No route from stored readings to ISO velocity units

The problem

ISO 10816-3 grades machines in mm/s RMS. The platform stores features in `scaled_eng`. Without a conversion, the standard's limits can be printed but never applied.

The fix

A conversion step that integrates the 1X acceleration at the measured shaft speed to give velocity. The mechanism now exists and is unit-tested.

Why it still refuses to produce a single answer

It will not guess what the stored numbers physically are, and I can now show that this refusal is correct rather than merely cautious. `feature_extraction.py` takes the CSV column exactly as supplied — its own docstring says "*scaled eng. units from CSV*". The sensor's sensitivity (100 mV/g, stored on the sensor configuration) is read only by `acquisition_config.py` to set up the DAQ hardware; it never touches the feature path. **The platform therefore never records whether a stored value is volts or g**, and the two differ by a factor of ten.

Result

Read as g: **34.4 mm/s**. Read as volts: **343.7 mm/s**. The ISO boundary above which vibration is expected to damage a machine is 7.1–11.0 mm/s. Both readings are implausible — a pump genuinely running at either would be audibly failing. The report prints the figure with an explicit warning and **assigns no severity zone**.

Status: the mechanism is fixed; the answer stays blocked by item 5 below. The most likely explanation is that the source file is synthetic test data rather than a real measurement — which real captures would settle immediately.

Not fixed — the three that need a person

NEEDS A HUMAN 3. The only baseline is a copy of the measurement

The gap is narrower than it first appears. The 144 unassessed readings come from *exactly two* features — `envelope_rms` (72) and `fft_band_energy_0_500` (72). Both are `rule_type = percent_baseline`, the only rule type that needs a baseline at all. Two features × 8 channels × 9 captures = 144. The **other 576 assessments never needed a baseline** and already work.

A baseline file also genuinely exists, with real data in it: 8 channels, 8192 samples, 25600 Hz. It is simply marked `is_primary = false` and has zero computed features stored. Both of those are software operations — no plant visit is required to change them.

So why is this still listed as needing a person? Because that baseline is `sample_vibration_data (1).csv` — **the same file as the captures being measured against it**. Mark it primary and compute its features, and every reading becomes exactly 100% of baseline, which is below the 120% normal limit. All 144 would come back **"normal"**.

That is *worse* than leaving them unassessed. `no_baseline` honestly reports *not checked*. "Normal" would report *checked and healthy* — a false all-clear produced purely by comparing a file to itself. It would be the single most misleading thing this report could say.

What is needed: a capture from a genuinely different, known-healthy run, marked primary. The obstacle is not the software; it is that no such capture exists yet.

Evidence: all 144 `no_baseline` rows are `envelope_rms` and `fft_band_energy_0_500`, both `percent_baseline`; `sensor_baselines` → 1 row, `is_primary = f`, 8 channels / 8192 samples, file `sample_vibration_data (1).csv`; `baseline_channel_features` → 0 rows; `threshold_evaluator.py` returns `STATUS_NO_BASELINE` only for `percent_baseline`.

NEEDS A SCHEMA CHANGE**4. Thresholds cannot be scoped to a sensor**

This is the finding I would most want reviewed. `feature_threshold_rules` has **no `sensor_id` column at all** — only `feature_code`, `machine_type` and `channel`. All 10 active rules are set to `machine_type = <any>` and `channel = <all>`, so one global number is applied to every machine on the platform.

The effect is visible immediately in the RMS feature:

QUANTITY	VALUE	MEANING
Critical limit (global rule)	0.02	applies to every machine
Lowest RMS reading recorded	0.024	already above the limit
Highest RMS reading recorded	0.424	21 × the limit
RMS readings marked critical	72 of 72	100%

A threshold that *no reading has ever satisfied* is describing its own configuration, not the machine.

What scoping is available, and what actually works

This is worth stating precisely, because the three levels behave differently:

SCOPE	IN THE SCHEMA?	HONOURED WHEN FEATURES ARE COMPUTED?	RULES SET
Per channel	Yes — <code>channel</code> column	Yes. <code>resolve_rule()</code> takes a channel override and falls back to the global rule	0
Per machine type	Yes — <code>machine_type</code> column	No. <code>get_active_threshold_rules()</code> supports it, but the feature pipeline calls <code>get_resolved_rule_map(db)</code> without a machine type, so a Pump-specific rule would be silently ignored	0
Per sensor / machine	No such column	Not possible	—

So per-channel limits could be set today with no code change, and nobody has set any. Machine-type limits need a small code fix first — passing the machine type into the rule lookup, which the equipment join in item 6 already makes available. **Neither reaches per-machine scoping**, which needs a migration: a generic limit says "unusual for pumps in general", never "unusual for *this* pump".

In the meantime the report already refuses to present these statuses as findings about the equipment.

Evidence: column list of `feature_threshold_rules`; all 10 active rules read back with null machine type and null channel (0 machine-type rules, 0 channel rules); 72 RMS rows spanning 0.02403 to 0.42409, all with `status = 'critical'`; `feature_storage.persist_upload_features_and_trends()` resolves rules by channel only.

NEEDS REAL DATA 5. The nine captures are one capture, ingested nine times

Fingerprinting each upload's complete feature vector gives **9 uploads but only 1 distinct vector**. Every capture carries numerically identical values. The files are `sample_vibration_data (1).csv` (7 times) and `sample_vibration_data.csv` (twice).

Consequence: no trend, no rate of change, and no prediction is possible. Nothing here is a history. The report shows trend columns as *"not assessable"* rather than flat, because a flat line would be an artefact of repeated ingestion that a reader could easily mistake for a stable machine.

What is needed: genuine captures taken at different times. This would also settle item 2 — whether the implausible velocity is a scaling fault or simply an artefact of synthetic test data.

Evidence: MD5 over each upload's full ordered feature vector → 9 uploads, 1 distinct fingerprint.

Correction — an earlier finding of mine was wrong

CORRECTION 6. The machine profile exists after all

I previously reported that *"no machine profile is registered for CWP-001, so the bearing geometry needed for BPFO/BPFI is unknown."* **That was incorrect**, and I would rather correct it here than have it carried forward into planning.

`equipment_masters` holds a complete record for CWP-001, and `sensor_configurations.equipment_id` joins this report's sensor directly to it:

FIELD	VALUE	WHAT IT UNLOCKS
Rated power	75 kW	the <code>--power-kw</code> flag the agent asks for
Rated speed	1480 rpm	independent check on the estimated speed
Foundation	Concrete Foundation	the <code>--foundation</code> flag
Bearing DE	6312-C3	drive-end fault frequencies
Bearing NDE	6310-C3	non-drive-end fault frequencies

So this is an **integration gap in the report agent, not a gap in the data**. The agent currently asks the operator to supply power, foundation and driver arrangement on the command line when all of it is already in the database, one join away.

Computed now, from that live record at the measured 1500 rpm, this is what connecting it would produce:

BEARING	BPFO	BPFI	BSF	FTF
6312-C3 (drive end)	87.6 Hz (3.51X)	137.4 Hz (5.49X)	53.8 Hz	9.74 Hz
6310-C3 (non-drive end)	77.5 Hz (3.10X)	122.5 Hz (4.90X)	52.7 Hz	9.69 Hz

Caveat, raised by the domain library itself and not by me: these are estimated from ISO 15 standard boundary dimensions, not from a manufacturer drawing. Confidence 0.6. The ball count may be off by one, which would shift BPFO and BPFI by roughly 10%. Adding the real geometry for 6312 and 6310 to the bearing catalogue would raise that to full confidence.

Recommended order of work

1. **Connect the report agent to `equipment_masters`** . Code only, no plant access, no schema change. Removes three command-line flags, adds bearing fault frequencies, and cross-checks the shaft speed against the nameplate automatically. Cheapest item on this list by a wide margin.
2. **Ingest genuine, distinct captures**. Unblocks all trend and prediction work, and settles whether the implausible velocity figure is a real scaling fault or an artefact of test data.
3. **Capture and mark a healthy baseline**. Converts 144 unassessed readings into real assessments. Note that marking the *existing* baseline primary would clear all 144 to "normal" within a minute and would be actively misleading, because it is the same file as the measurements. This needs a genuinely different healthy capture, from someone who can vouch for the machine's condition at the time.
4. **Pass the machine type into the threshold lookup**. A few lines, once item 1 is done: it makes the existing `machine_type` column functional. Per-channel rules already work today and none have been set.
5. **Add sensor scoping to `feature_threshold_rules`** . The largest change, needing a migration and UI. Until then, threshold statuses describe configuration rather than equipment.
6. **Add real 6312 / 6310 geometry to the bearing catalogue**. Small, and lifts the bearing frequencies above from estimated to catalogue confidence.

How these conclusions were checked

Every figure in this document was read from the running platform database or computed by the unit-tested domain library at the time of writing. None of it was recalled from earlier in the work. Specifically: threshold rules and their scope, RMS reading range and status counts, baseline row and its feature count, capture fingerprints, the equipment record and its join to the sensor were all queried directly; the bearing fault frequencies were computed by `app/domain/bearing.py` ; the unit finding was established by reading the feature-extraction source.

This re-check is the reason item 6 appears as a correction. The claim had been carried forward from an earlier stage of the work and did not survive verification against the database.

Report agent — data readiness findings for CWP-001 · prepared 2026-09-08.

The report agent produces Markdown to the terminal and is not wired into the dashboard. Two data problems fixed in code, three requiring plant access, one earlier finding corrected.